# 计网第三次实验：配置Web服务器，捕获HTTP报文并分析

2312322 邹佳衡 密码与网络空间安全学院


- 计网第三次实验：配置Web服务器，捕获HTTP报文并分析
  - 实验要求
  - 实验工具
  - web服务器搭建
  - 进行wireshark抓包
  - 分析实验结果
    - 总体分析
    - 以某个连接流进行分析
    - 以某几条报文进行分析
  - 总结


## 实验要求

(1) 搭建Web服务器（自由选择系统），并制作简单的Web页面，包含简单文本信息（包含专业、学号、姓名）和6幅图像。

(2) 通过浏览器获取自己编写的Web页面，使用Wireshark捕获浏览器与Web服务器的交互过程。

(3) 分析两个报文的封装层次，包括数据链路层、互连层、传输层、应用层。

(4) 对整个HTTP交互过程进行详细说明，使用Wireshark过滤器使其仅显示HTTP协议。

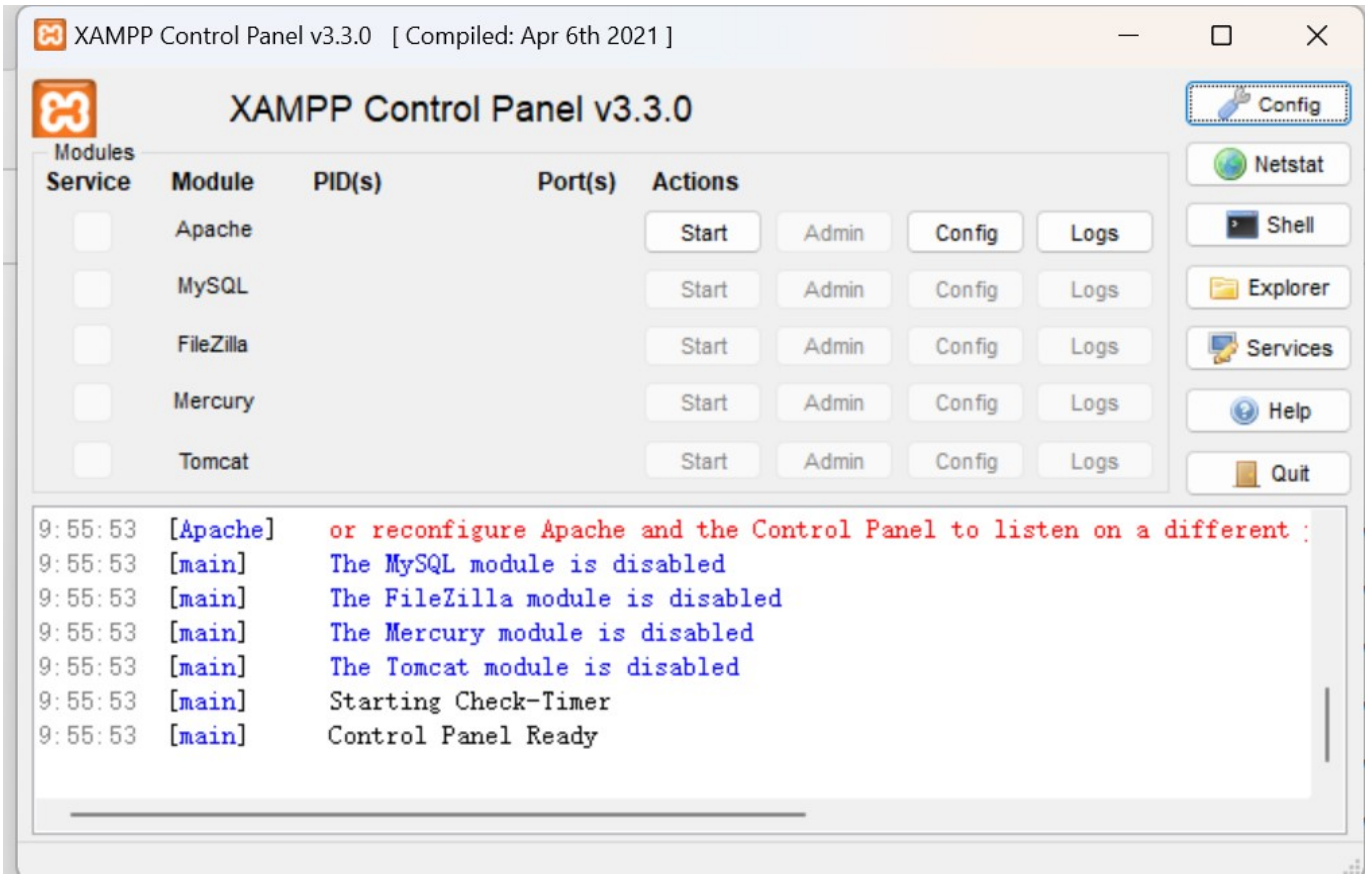
(5) 提交HTML文档、Wireshark捕获文件和实验报告

注：页面不要太复杂，包含所要求的基本信息即可。使用HTTP，不要使用HTTPS。

## 实验工具

**xampp**

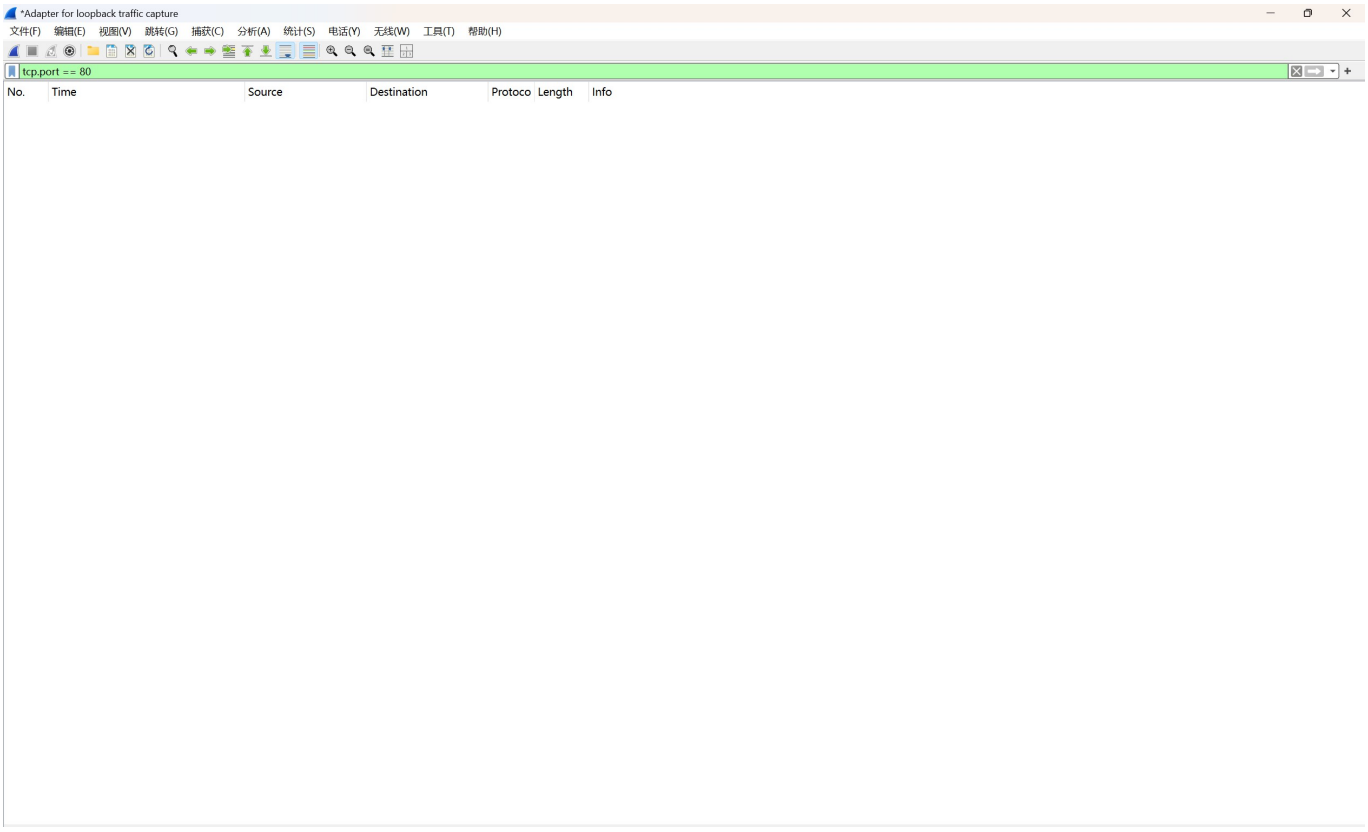
XAMPP 是一个免费开源的跨平台Web服务器解决方案包，其内部包含apache（Web服务器），MariaDB（数据库，MySQL分支），PHP（脚本语言），Perl（脚本语言）等多种功能和服务。




**wireshark**

Wireshark 是一款开源的网络协议分析工具，前身为 Ethereal。它能够捕获网络数据包，并以人性化的方式展示数据包的详细内容，是网络工程师、安全分析师和开发人员必备的工具。

在本次实验中，我们主要通过wireshark获取数据包传输信息，在实验时，要使用本地回环作为捕获过滤器，以获取我们本地服务器的数据传输记录，使用tcp.port == 80作为显示过滤器，只看http相关流量。



# web服务器搭建

我们到网上下载六张图片到本地，当然也可以直接使用网络图片，但为了编程方便我选择用本地图片。

我们需要设计一个简单的html文件，内容要包括自己的个人信息以及六张图片，具体代码如下：

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>HTTP协议分析测试</title>
</head>
<body>
    <h1>HTTP协议分析实验</h1>
    <p><strong>专业：</strong>密码与网络空间安全学院</p>
    <p><strong>学号：</strong>2312322</p>
    <p><strong>姓名：</strong>邹佳衡</p>

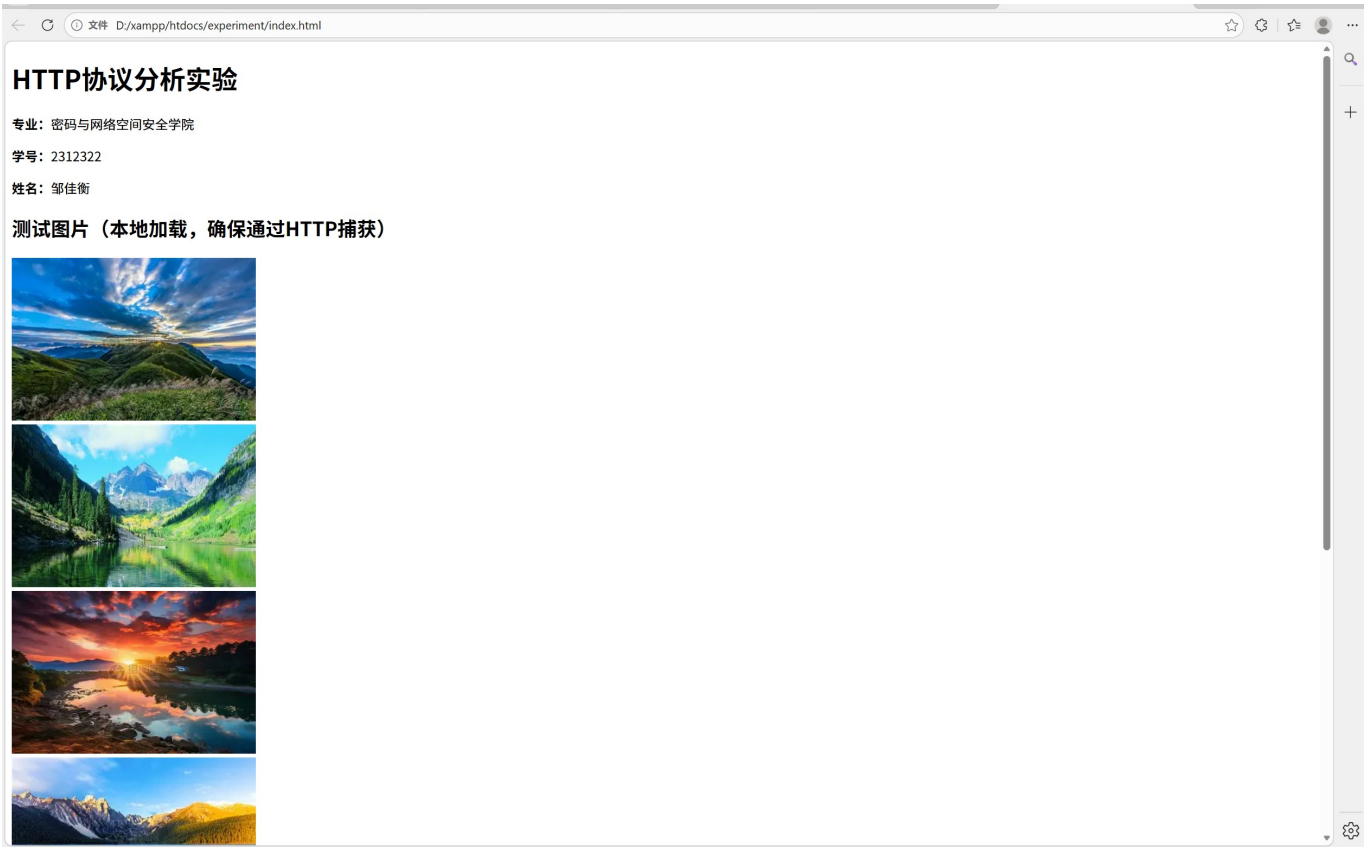
    <h2>测试图片（本地加载，确保通过HTTP捕获）</h2>

    <!-- 使用本地图片，文件名为 image1.jpg 到 image6.jpg -->
    <img src="images/image1.jpg" alt="测试图片1" width="300" height="200">
    <br>
    <img src="images/image2.jpg" alt="测试图片2" width="300" height="200">
    <br>
    <img src="images/image3.jpg" alt="测试图片3" width="300" height="200">
    <br>
    <img src="images/image4.jpg" alt="测试图片4" width="300" height="200">
    <br>
    <img src="images/image5.jpg" alt="测试图片5" width="300" height="200">
    <br>
    <img src="images/image6.jpg" alt="测试图片6" width="300" height="200">

    <p>访问时间：<span id="time"></span></p>

    <script>
        document.getElementById('time').textContent = new Date().toLocaleString();
    </script>
</body>
</html>
```

这是正常打开index.html的效果：




需要注意的是，我要把项目放在xampp文件夹的htdocs文件夹里，将图片放在images文件夹里，再将image文件夹和index.html文件放在htdocs的同一目录下，具体的情况如下：

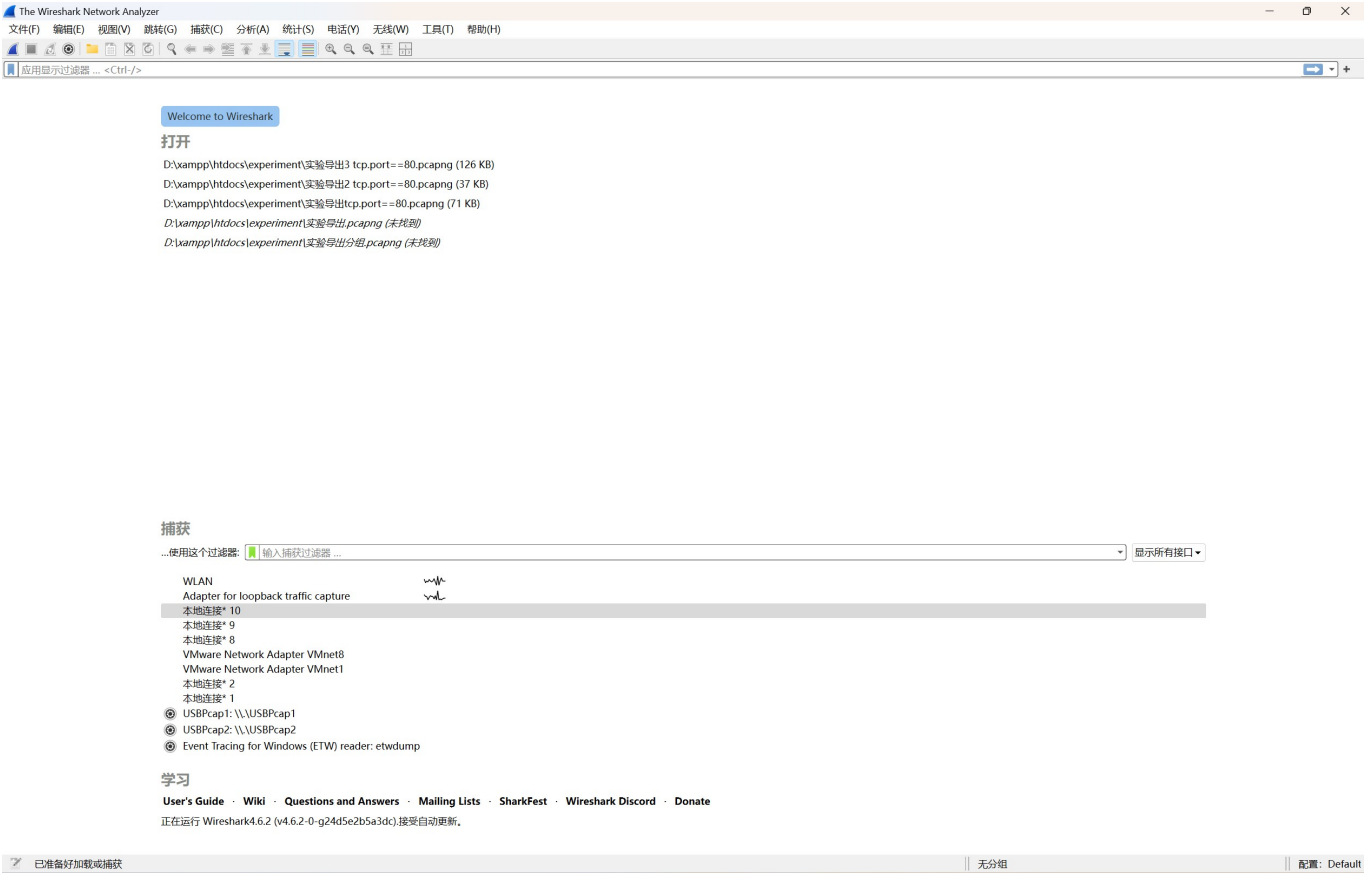
```
xampp\htdocs\experiment
                |-index.html
                |-images
                    |-image1.jpg
                    |-image2.jpg
                    |-image3.jpg
                    |-image4.jpg
                    |-image5.jpg
                    |-image6.jpg
```

然后，打开xampp的控制台，开始运行apache，我的apache是在端口80上监听，紧接着来到浏览器，清空所有缓存，并使用无痕模式，在网站上输入http://localhost:80/experiment/index.html，暂时不要先打开，等到wireshark开始运行后再打开。

之所以要清空缓存，是为了防止网站直接加载缓存的图片，这样我们就看不到六次tcp请求了，可能只有一两次。

# 进行wireshark抓包

接下来，我们开始配置wireshark，在打开wireshark后，我们会看到这样的页面：




这里让我们选择捕获过滤器，我们选第二个，也就是Adapter for loopback这个，这是专门用于捕获本地回环流量的适配器，我们访问的是http://localhost，这是回环流量，使用此过滤器可以排除大部分噪声。

然后，在进入wireshark后，等到开始进行捕获时，再进入我们的网站，等到网站完全加载完成后，暂停wireshark捕获，在上方输入框中输入tcp.port == 80，这是显示过滤器，进一步筛选http流量，确认无误后保存即可。

# 分析实验结果

## 总体分析

在经历了一些波折后，我们成功获得了符合实验需要的wireshark捕获记录：

| No. | Time | Source | Destination | Protoco | Length | Info |
|---|---|---|---|---|---|---|
| 195 | 2026-01-01 10:02:07.037294 | ::1 | ::1 | TCP | 64 | 23715 → http(80) [ACK] Seq=1 Ack=1 Win=263424 Len=0 |
| 239 | 2026-01-01 10:02:07.096676 | ::1 | ::1 | TCP | 64 | 23715 → http(80) [ACK] Seq=1329 Ack=17353 Win=263424 Len=0 |
| 241 | 2026-01-01 10:02:07.096692 | ::1 | ::1 | TCP | 64 | 23715 → http(80) [ACK] Seq=1329 Ack=17889 Win=262912 Len=0 |
| 601 | 2026-01-01 10:02:12.109695 | ::1 | ::1 | TCP | 64 | 23715 → http(80) [ACK] Seq=1329 Ack=17890 Win=262912 Len=0 |
| 200 | 2026-01-01 10:02:07.037930 | ::1 | ::1 | TCP | 64 | 23715 → http(80) [ACK] Seq=696 Ack=1513 Win=263424 Len=0 |
| 193 | 2026-01-01 10:02:07.037225 | ::1 | ::1 | TCP | 76 | 23715 → http(80) [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM |
| 219 | 2026-01-01 10:02:07.091579 | ::1 | ::1 | TCP | 64 | 34702 → http(80) [ACK] Seq=1 Ack=1 Win=263424 Len=0 |
| 311 | 2026-01-01 10:02:07.105781 | ::1 | ::1 | TCP | 64 | 34702 → http(80) [ACK] Seq=1250 Ack=32935 Win=263424 Len=0 |
| 318 | 2026-01-01 10:02:07.105811 | ::1 | ::1 | TCP | 64 | 34702 → http(80) [ACK] Seq=1250 Ack=41103 Win=263424 Len=0 |
| 592 | 2026-01-01 10:02:12.109592 | ::1 | ::1 | TCP | 64 | 34702 → http(80) [ACK] Seq=1250 Ack=41104 Win=263424 Len=0 |
| 292 | 2026-01-01 10:02:07.102093 | ::1 | ::1 | TCP | 64 | 34702 → http(80) [ACK] Seq=634 Ack=9895 Win=263424 Len=0 |
| 217 | 2026-01-01 10:02:07.091549 | ::1 | ::1 | TCP | 76 | 34702 → http(80) [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM |
| 216 | 2026-01-01 10:02:07.091421 | ::1 | ::1 | TCP | 64 | 39180 → http(80) [ACK] Seq=1 Ack=1 Win=263424 Len=0 |
| 269 | 2026-01-01 10:02:07.100281 | ::1 | ::1 | TCP | 64 | 39180 → http(80) [ACK] Seq=634 Ack=9197 Win=263424 Len=0 |
| 602 | 2026-01-01 10:02:12.109698 | ::1 | ::1 | TCP | 64 | 39180 → http(80) [ACK] Seq=634 Ack=9198 Win=263424 Len=0 |
| 214 | 2026-01-01 10:02:07.091393 | ::1 | ::1 | TCP | 76 | 39180 → http(80) [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM |
| 210 | 2026-01-01 10:02:07.091087 | ::1 | ::1 | TCP | 64 | 47943 → http(80) [ACK] Seq=1 Ack=1 Win=263424 Len=0 |
| 261 | 2026-01-01 10:02:07.099885 | ::1 | ::1 | TCP | 64 | 47943 → http(80) [ACK] Seq=634 Ack=12414 Win=263424 Len=0 |
| 594 | 2026-01-01 10:02:12.109637 | ::1 | ::1 | TCP | 64 | 47943 → http(80) [ACK] Seq=634 Ack=12415 Win=263424 Len=0 |
| 208 | 2026-01-01 10:02:07.091047 | ::1 | ::1 | TCP | 76 | 47943 → http(80) [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM |
| 213 | 2026-01-01 10:02:07.091261 | ::1 | ::1 | TCP | 64 | 59096 → http(80) [ACK] Seq=1 Ack=1 Win=263424 Len=0 |
| 260 | 2026-01-01 10:02:07.099853 | ::1 | ::1 | TCP | 64 | 59096 → http(80) [ACK] Seq=634 Ack=12902 Win=263424 Len=0 |
| 597 | 2026-01-01 10:02:12.109669 | ::1 | ::1 | TCP | 64 | 59096 → http(80) [ACK] Seq=634 Ack=12903 Win=263424 Len=0 |
| 211 | 2026-01-01 10:02:07.091230 | ::1 | ::1 | TCP | 76 | 59096 → http(80) [SYN] Seq=0 Win=64800 Len=0 MSS=1440 WS=256 SACK_PERM |
| 201 | 2026-01-01 10:02:07.085431 | ::1 | ::1 | HTTP | 697 | GET /experiment/images/image1.jpg HTTP/1.1 |
| 206 | 2026-01-01 10:02:07.090836 | ::1 | ::1 | HTTP | 697 | GET /experiment/images/image2.jpg HTTP/1.1 |
| 220 | 2026-01-01 10:02:07.091701 | ::1 | ::1 | HTTP | 697 | GET /experiment/images/image3.jpg HTTP/1.1 |
| 222 | 2026-01-01 10:02:07.091756 | ::1 | ::1 | HTTP | 697 | GET /experiment/images/image4.jpg HTTP/1.1 |
| 224 | 2026-01-01 10:02:07.091853 | ::1 | ::1 | HTTP | 697 | GET /experiment/images/image5.jpg HTTP/1.1 |
| 226 | 2026-01-01 10:02:07.091889 | ::1 | ::1 | HTTP | 697 | GET /experiment/images/image6.jpg HTTP/1.1 |
| 196 | 2026-01-01 10:02:07.037448 | ::1 | ::1 | HTTP | 759 | GET /experiment/index.html HTTP/1.1 |
| 293 | 2026-01-01 10:02:07.105212 | ::1 | ::1 | HTTP | 680 | GET /favicon.ico HTTP/1.1 |
| 240 | 2026-01-01 10:02:07.096684 | ::1 | ::1 | HTTP | 600 | HTTP/1.1 200 OK (image/jpeg) |
| 257 | 2026-01-01 10:02:07.099782 | ::1 | ::1 | HTTP | 1445 | HTTP/1.1 200 OK (image/jpeg) |
| 259 | 2026-01-01 10:02:07.099790 | ::1 | ::1 | HTTP | 957 | HTTP/1.1 200 OK (image/jpeg) |
| 268 | 2026-01-01 10:02:07.100229 | ::1 | ::1 | HTTP | 620 | HTTP/1.1 200 OK (image/jpeg) |
| 283 | 2026-01-01 10:02:07.101100 | ::1 | ::1 | HTTP | 293 | HTTP/1.1 200 OK (image/jpeg) |
| 291 | 2026-01-01 10:02:07.102069 | ::1 | ::1 | HTTP | 1318 | HTTP/1.1 200 OK (image/jpeg) |
| 317 | 2026-01-01 10:02:07.105801 | ::1 | ::1 | HTTP | 1032 | HTTP/1.1 200 OK (image/x-icon) |
| 199 | 2026-01-01 10:02:07.037913 | ::1 | ::1 | HTTP | 136 | HTTP/1.1 200 OK (text/html) |

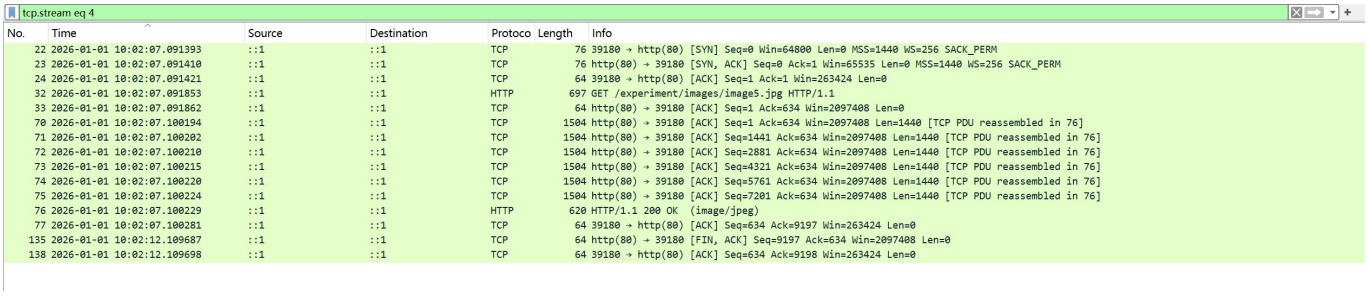
首先我们的捕获从2026-01-01 10:02:07.037225一直到2026-01-01 10:02:12.109698，大约5秒的时间，全部都是iPv6本地回环通讯，也就是从::1到::1，只有一个http服务器，端口号为80，有多个客户端，端口号包括23715, 5676, 47943等。

然后，我们注意到总共有六个连接流，我们可以通过显示过滤器输入tcp.stream eq n来查看第n个连接流，以下是各个连接流内容：

| n | 服务器（apache）端口 | 客户端（浏览器）端口 | 涉及内容 |
|---|---|---|---|
| 0 | 80 | 23715 | index.html+image1.jpg |
| 1 | 80 | 5676 | image2.jpg |
| 2 | 80 | 47943 | image3.jpg |
| 3 | 80 | 59096 | image4.jpg |
| 4 | 80 | 39180 | image5.jpg |
| 5 | 80 | 34702 | image6.jpg+favicon.ico |

## 以某个连接流进行分析

接下来我们逐层分析，以image5.jpg为例，这个比较简单：




这是一次基于IPv6本地回环地址（::1）的HTTP图像请求与响应的完整TCP连接过程，捕获于本地环回接口。整个过程包含TCP三次握手、HTTP请求/响应、TCP四次挥手，是典型的HTTP 1.1短连接会话。总体信息如下：

- 通讯双方：客户端（端口39180）与本地HTTP服务器（端口80）
- 协议：IPv6 + TCP + HTTP
- 传输内容：图像文件 /experiment/images/image5.jpg
- 总响应数据量：9196字节（通过7个TCP段传输）

首先，我们先来总结一下整个连接与传输过程：

### 1.tcp三次握手

这里在前两次实验已经说过很多次了，就简单一提，首先由浏览器向本地服务器发起连接请求，然后由服务器确认连接请求并同步自身序列号，最后客户端确认，握手完成，连接建立。

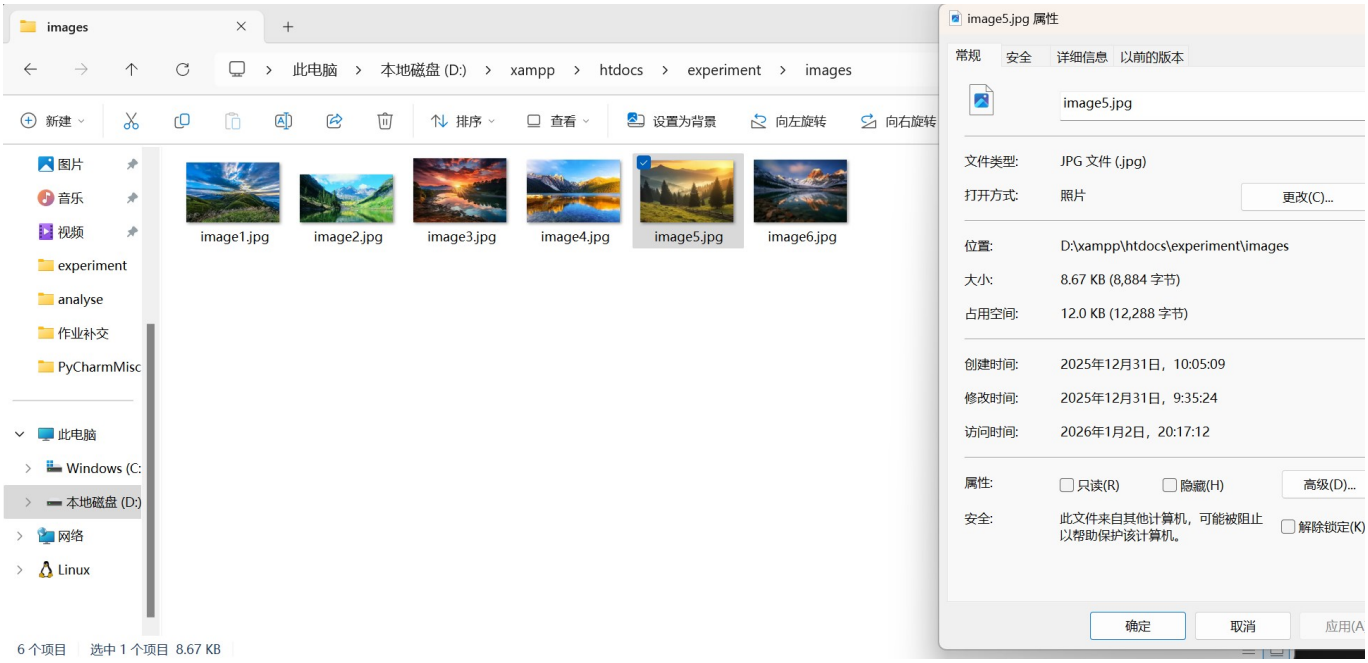
### 2.http请求

首先是由浏览器发出的向服务器的http请求：*GET /experiment/images/image5.jpg HTTP/1.1*，其中包含了HTTP请求头，Length=633字节，包含请求方法、路径、协议版本及头部信息。

然后是服务器回复的确认收到请求的数据包。ACK，Ack=634这个。

### 3.http响应

服务器将响应分为7个TCP段传输，前六条是由服务器向浏览器发送的图像数据，每段均为 ACK报文头 + 1440 字节数据，连续发送，间隔极短（4–8微秒），体现本地回环的高速性。然后是服务器发送的一个携带HTTP响应头+剩余数据共556字节，标明内容类型为JPEG图像。tcp传输的内容总共9196字节，而tcp传输包含两部分，http头部和JPEG图像数据，http头部共312字节，减去后图像数据为8884字节，刚好和image5.jpg大小一致。




传输好后，浏览器再给服务器一个确认，ACK，Ack=9197，代表客户端确认收到全部响应数据。

### 4.tcp四次挥手

这个也不用赘述，但是我们注意到四次挥手只发现了两次，就是服务器主动发起和浏览器响应，因为这是本地抓包且为单向关闭，后续可能缺少了两步。

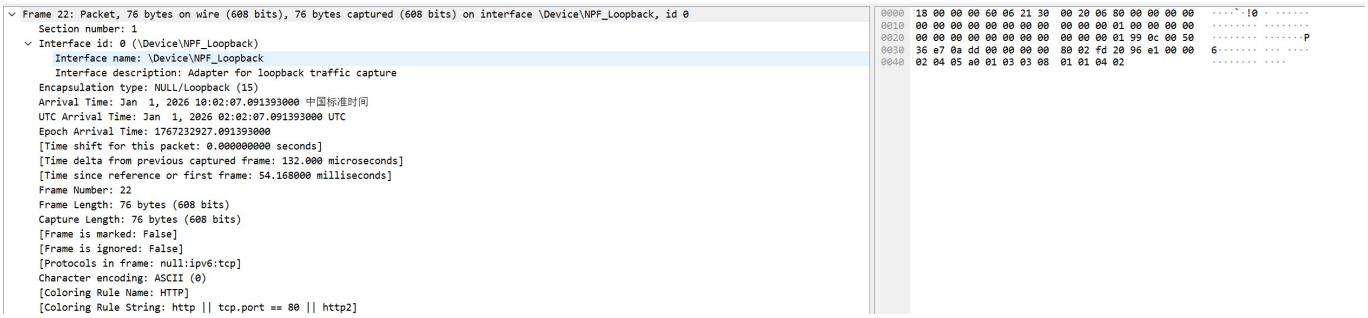
## 以某几条报文进行分析

在这次实验中，完整的封装层次如下：

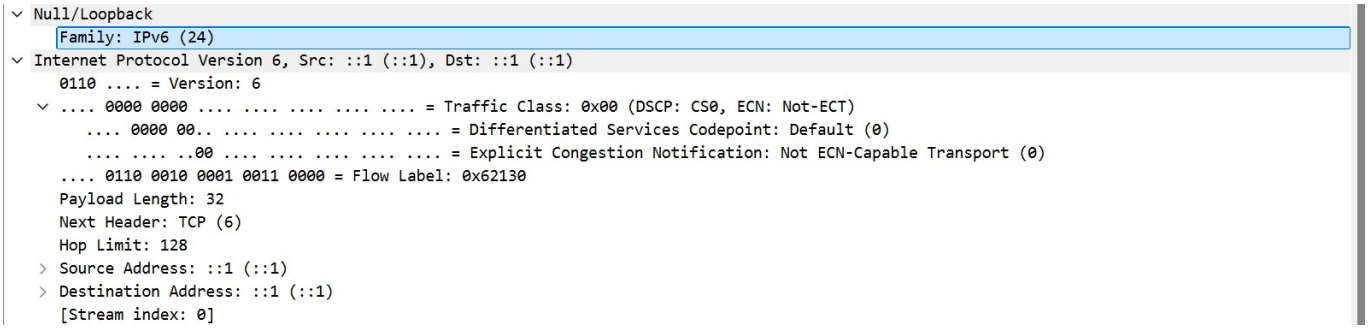
- 应用层：为应用程序提供网络服务接口，实现具体的应用功能。本次实验中主要是http协议。
- 传输层：提供端到端的可靠/不可靠数据传输服务。本次实验中主要是tcp协议。
- 网络层：实现主机到主机的逻辑寻址和路由选择。本次实验中主要是IPv6本地回环。
- 数据链路层：实现相邻节点间的可靠帧传输。本次实验中主要是环回接口特殊封装。

接下来来详细讲解具体报文的封装，以几个典型的报文为例：

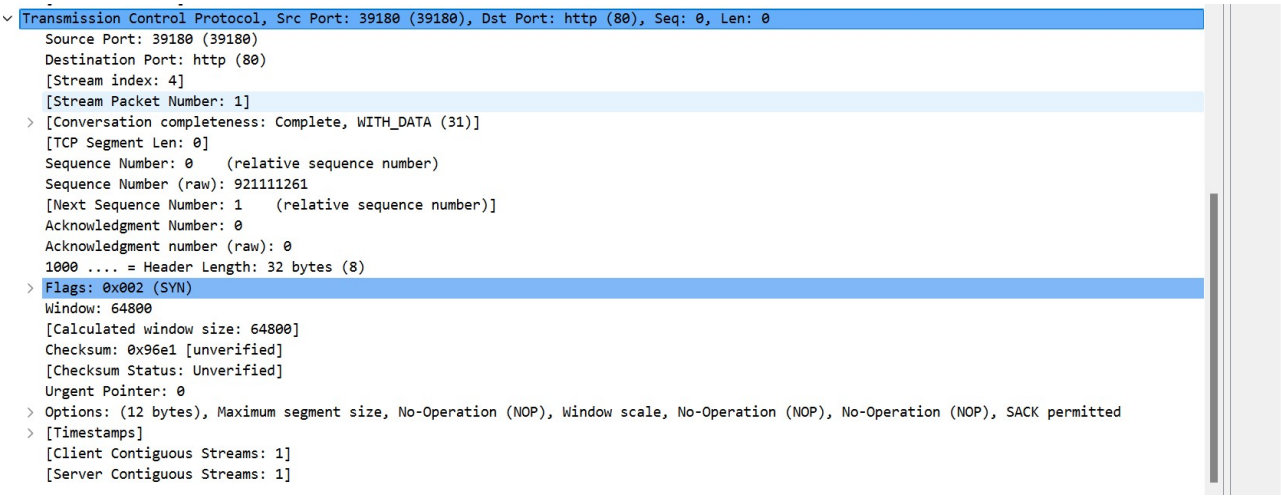
### No.22 tcp SYN包



在数据链路层中，它标识了封装类型为NULL/Loopback (15)，这是环回接口专用虚拟封装，此外还有一些长度、编码等信息。



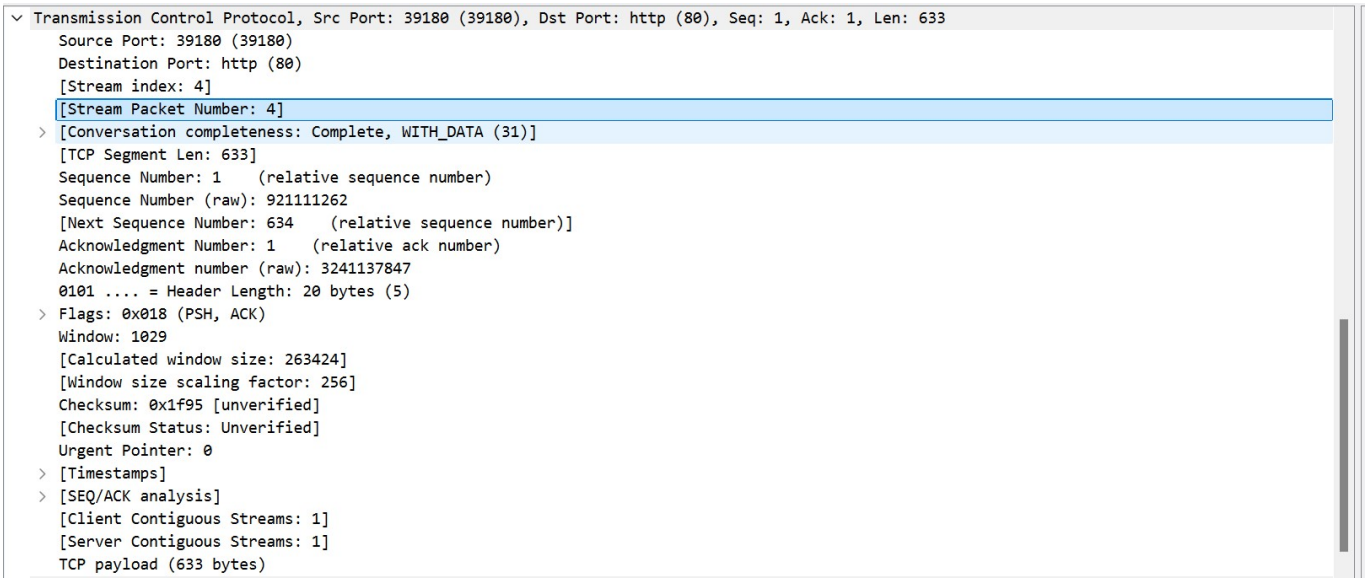

在网络层中，它标识了协议版本为iPv6，源地址和目的地址都是::1，iPv6报文头部总长度为40字节。




在传输层中，它标识了源端口和目的端口，序列号seq为0，标志位为SYN，接收窗口大小为64800。

### No.32 http GET请求

数据链路层和网络层封装与第一个基本一致。




在传输层，实现了三次握手后，序列号加一变为一，确认号也为一，数据长度为633个字节，代表http请求内容长度。标志位为PSH+ACK。

```
Hypertext Transfer Protocol
  > GET /experiment/images/image5.jpg HTTP/1.1\r\n
    Host: localhost\r\n
    Connection: keep-alive\r\n
    sec-ch-ua-platform: "Windows"\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0\r…
    sec-ch-ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"\r\n
    sec-ch-ua-mobile: ?0\r\n
    Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8\r\n
    Sec-Fetch-Site: same-origin\r\n
    Sec-Fetch-Mode: no-cors\r\n
    Sec-Fetch-Dest: image\r\n
    Referer: http://localhost/experiment/index.html\r\n
    Accept-Encoding: gzip, deflate, br, zstd\r\n
    Accept-Language: zh-CN,zh;q=0.9\r\n
    \r\n
    [Response in frame: 76]
    [Full request URI: http://localhost/experiment/images/image5.jpg]
```

在应用层，标识使用的是http的GET方法获取资源experiment/images/image5.jpg。协议版本http 1.1，头部字段包括Host：localhost（虚拟主机标识）、User-Agent：客户端标识、Accept：可接受的内容类型、Connection：keep-alive（保持连接）等。

**No.76 http响应**

数据链路层和网络层封装同上。

```
Transmission Control Protocol, Src Port: http (80), Dst Port: 39180 (39180), Seq: 8641, Ack: 634, Len: 556
    Source Port: http (80)
    Destination Port: 39180 (39180)
    [Stream index: 4]
    [Stream Packet Number: 12]
  > [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 556]
    Sequence Number: 8641    (relative sequence number)
    Sequence Number (raw): 3241146487
    [Next Sequence Number: 9197    (relative sequence number)]
    Acknowledgment Number: 634    (relative ack number)
    Acknowledgment number (raw): 921111895
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x018 (PSH, ACK)
    Window: 8193
    [Calculated window size: 2097408]
    [Window size scaling factor: 256]
    Checksum: 0x8022 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [Timestamps]
  > [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
    TCP payload (556 bytes)
    TCP segment data (556 bytes)
```

包括序列号8641（前6段共传输8640字节），确认号634（确认客户端最后的序列号），数据长度556字节（最后一段HTTP数据）。

```
Hypertext Transfer Protocol
  > HTTP/1.1 200 OK\r\n
    Date: Thu, 01 Jan 2026 02:02:07 GMT\r\n
    Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12\r\n
    Last-Modified: Wed, 31 Dec 2025 01:35:24 GMT\r\n
    ETag: "22b4-647357d7703bb"\r\n
    Accept-Ranges: bytes\r\n
  > Content-Length: 8884\r\n
    Keep-Alive: timeout=5, max=100\r\n
    Connection: Keep-Alive\r\n
    Content-Type: image/jpeg\r\n
    \r\n
    [Request in frame: 32]
    [Time since request: 8.376000 milliseconds]
    [Request URI: /experiment/images/image5.jpg]
    [Full request URI: http://localhost/experiment/images/image5.jpg]
    File Data: 8884 bytes
  > [Expert Info (Note/Malformed): HTTP body subdissector failed, trying heuristic subdissector]
```

包括状态行HTTP/1.1 200 OK，代表已经传完数据。

此外还有一个特殊标记，Wireshark将JPEG数据识别为Media Type，并在协议栈中显示media。

# 总结

本次实验通过搭建本地Web服务器并访问包含个人信息和6张图片的网页，使用Wireshark成功捕获了浏览器与服务器之间的完整交互过程。分析发现，在HTTP/1.1协议下，浏览器为提升加载性能采用了多连接并行下载策略——主HTML页面与第一张图片共用同一TCP连接，其余5张图片各启用独立连接，网站图标则共用了最后一个图片连接。实验清晰地展示了从TCP三次握手、HTTP请求响应、数据分片传输到连接保持的全过程，验证了HTTP协议的无状态性、TCP的可靠传输机制以及浏览器在本地环境下的优化行为，为理解Web应用层与传输层协同工作原理提供了实证基础。